Módulo `console`

> Código: `src/console.h`, `src/console.cpp` · Relacionados: 07 — Protocolo e interfaces, main, settings

Papel no sistema

Interface pela porta serial (USB, 115200 baud): interpretador de comandos (`diag`, `calib`, `get`, `set`, `motor`, `plot`, `reset`, `help`), compatibilidade com os comandos do BioSpine V3 (`START`, `STOP`, `STATUS`, `CALIBRATE`, respostas em JSON) e as saídas contínuas (Serial Plotter e JSON a 1 Hz).

É a ferramenta de bancada: o `diag` existe para quem monta o hardware descobrir, sem mexer em código, qual fio está errado. A regra combinada é "se o `diag` dá ❌, o problema é fio, não código".

Dependências

- Usa: API `app_*` (main), `sensores()` e `sensores_scanI2C` (sensors), `configPadrao`, `modoNome`, `modoDeTexto` (settings), `web_ativo`, `web_ip`, `web_clientes` (web), config.
- É usado por: `main.cpp` (`console_iniciar`, `console_atualizar`, `console_enviarDados`, `console_evento`, `console_respostaJson`).
- Roda em: **loop** (nunca na tarefa de controle).

Tipos e constantes

Variável	Tipo	Descrição
`linha[256]`	char[]	linha sendo digitada, até o Enter
`tam`	uint8_t	caracteres guardados em `linha`
`plot`	bool	`plot on` ativo: imprime dados a 10 Hz
`streamJson`	bool	`START` ativo: imprime o JSON do BioSpine a 1 Hz
`ultimoJson`	uint32_t	`millis()` do último JSON contínuo
`SIM`, `NAO`	const char*	✅ e ❌ em UTF-8 escritos byte a byte

Funções

`void console_iniciar()`

- **O que faz:** imprime a dica de uso e, se não houver calibração, o lembrete ("vista o colete... aperte o botão").
- **Chamada por:** `setup`.

`void console_atualizar(uint32_t agoraMs)`

- **O que faz:** lê os caracteres disponíveis **sem bloquear** e, a cada `\r` ou `\n`, executa a linha (`executar`). Linha vazia é ignorada (aceita `\r\n`). Mais de 255 caracteres: o excesso é descartado.
- **Por quê sem bloquear:** `Serial.readStringUntil` travaria o loop até o timeout (1 s), atrasando o Wi-Fi.
- **Chamada por:** `loop`, a cada volta.

`void console_enviarDados(const Status& s, uint32_t agoraMs)`

- **O que faz:** saídas contínuas, chamada a 10 Hz pelo loop.
  - `plot: desvio:..,limite:..,pitch:..,roll:..,flexPct:..,limiteFlex:..,alerta:..` (formato do Serial Plotter do Arduino e do `tools/grafico_serial.py`). `alerta` vale 5 ou 0 para aparecer como degrau visível na mesma escala dos graus.

- `streamJson`: uma linha JSON do BioSpine por segundo.

- **Chamada por:** `loop`.

`void console_evento(const char* tag, const char* msg)`

- **O que faz:** imprime `[tag] msg`.
- **Chamada por:** `drenarEventos` (eventos da tarefa de controle) e `app_calibrar`.

`void console_respostaJson(bool ok, const char* msg)`

- **O que faz:** imprime `{"success":true|false,"message":"..."}` (formato do BioSpine V3).
- **Chamada por:** `executar` (`START`, `STOP`, `CALIBRATE` sem MPU) e `drenarEventos` (`EVT_JSON_CALIB_OK`: "Calibration completed").

`static void executar(char* cmd)` — interpretador

1. Compara a linha **exata** com `START`, `STOP`, `STATUS`, `CALIBRATE` (maiúsculos, BioSpine V3).
2. Converte o resto para minúsculas e separa em até 3 palavras (`strtok` por espaço): comando, `a1`, `a2`.
3. Despacha: `help`/`ajuda`/`?`, `diag`, `calib`/`calibrar`, `get`, `set`, `motor`, `plot`, `status`, `reset`; senão "comando desconhecido".

- **Observação:** modifica a própria string (`tolower` e `strtok`).

`static void comandoSet(char* nome, char* valor)`

- **O que faz:** `set <nome> <valor>`: copia a config atual, troca um campo e chama `app_definirConfig` (que valida, salva na NVS e avisa o app). Depois imprime "salvo." e a config (`mostrarConfig`).
- **Conversão:** `atof` (texto não numérico vira 0 e depois é preso na faixa por `validarParams`); `tempo` negativo vira 0; `motor` preso em 0–100; `flex`/`flex2` aceitam `on`/`off`/`auto`.

`static void mostrarConfig()`

- **O que faz:** comando `get`: parâmetros, modo e situação de cada flex (`descreverSensor`) e calibração.

`static const char* descreverSensor(ModoSensor modo, bool detectado, bool ativo)`

- **O que faz:** texto da situação de um flex: `desligado`, `ativo`, `não detectado` ou `detectado, recalibre para usar` (ligado depois da calibração, ainda sem referência).

`static void diag()`

- **O que faz:** testa cada componente e diz o que conferir quando falha. Linhas, em ordem: I2C scan, MPU (ângulos, endereço, `WHO_AM_I`, offset do giroscópio), flex 1 (com o motivo provável da falha pela faixa da leitura), flex 2, FSR, botão, motor (pulsa 1 s), Wi-Fi AP (IP e clientes), calibração.
- **Observação:** lê `sensores()` sem trava (diagnóstico; um valor de um passo atrás não importa).

`static void jsonBioSpine(const Status& s)`

- **O que faz:** imprime `{"angle":..,"flex_left":..,"flex_right":..,"pressure":..,"status":"GOOD|WARNING|BAD","alert":..,"calibrated":..}`. `angle` = desvio, `flex_left` = flex 1, `flex_right` = flex 2, `pressure` = FSR, `status` = ok/atenção/alerta (indexado pelo valor do enum `Estado`).

`static void ajuda()`

- **O que faz:** imprime a lista de comandos (`help`).

Armadilhas e cuidados

- **`nomes[s.estado]` em `jsonBioSpine`** depende da ordem do enum `Estado` (0 = ok, 1 = atenção, 2 = alerta). Mudar o enum quebra o JSON do BioSpine.
- **Comandos do BioSpine só em maiúsculas exatas.** `start`, `stop` e `calibrate` minúsculos caem no interpretador normal e dão "comando desconhecido" (`status` minúsculo funciona).
- **`CALIBRATE` recusado por movimento não gera JSON** de falha: só a linha `[calib] recusada...`. Quem espera a resposta JSON do BioSpine fica sem ela nesse caso.
- **`plot` com qualquer argumento diferente de `off` liga** (`plot`, `plot on`, `plot x`).
- Texto novo com acento deve continuar em UTF-8 (o monitor serial do PlatformIO mostra certo).